UNIVERSIDAD DON BOSCO

DIRECCIÓN DE EDUCACIÓN A DISTANCIA

Diseño y Programación de Software Multiplataforma DPS941 G01T (Virtual)

DOCENTE:

Alexander Alberto Sigüenza Campos.

EXPERIENCIA DE APRENDIZAJE NÚMERO TRES

Poyecto React I

GRUPO NÚMERO 6

Integrantes:

Víctor Emmanuel Velasco Martínez - VM251307
Fernando José Gómez Hernández - GH251230
José Eduardo Aquino Medrano - AM252078
Milton Adalberto Ayala Ventura - AV251709
William Eduardo Montano Aguilar - MA251192

ENLACE DEL REPOSITORIO:

https://github.com/VMz69/project-manager

ENLACE DE SITIO DESPLEGADO:

pendiente

FECHA DE ENTREGA:

29 de marzo del 2026

Introducción

Este proyecto consiste en el desarrollo de una aplicación web para la gestión de proyectos y tareas utilizando React y Next.js, con el propósito de aplicar conceptos fundamentales como el manejo de estado, consumo de APIs, autenticación básica y organización modular del código.

El sistema permite administrar proyectos y asignar tareas a usuarios, incorporando roles diferenciados (gerente y usuario), lo que simula un entorno real de trabajo colaborativo. Además, se implementa una simulación de API REST mediante una capa de servicios que utiliza datos mock, permitiendo realizar operaciones CRUD sobre proyectos y tareas.

La aplicación está diseñada como un dashboard interactivo, enfocado en la simplicidad, usabilidad y una experiencia de usuario clara, facilitando la visualización del progreso y la gestión eficiente de actividades.

Objetivos

Objetivo General

Desarrollar una aplicación web para la gestión de proyectos y tareas que implemente autenticación básica, control de roles y consumo de APIs simuladas utilizando Next.js.

Objetivos Específicos

- Implementar registro e inicio de sesión de usuarios.
- Gestionar autenticación mediante localStorage.
- Crear y administrar proyectos.
- Crear, asignar y actualizar tareas.
- Implementar roles (gerente y usuario).
- Consumir datos a través de capa de servicios que simula una API REST usando datos mock.
- Aplicar buenas prácticas en la estructura del proyecto.
- Desarrollar una interfaz responsiva y funcional.

Tecnologías utilizadas

Para el desarrollo de este proyecto se utilizaron tecnologías modernas enfocadas en el desarrollo de aplicaciones web dinámicas y escalables. Estas herramientas permiten una mejor organización del código, reutilización de componentes y una integración sencilla con APIs.

- React
- Next.js
- TypeScript
- Simulación de API REST mediante servicios y datos mock
- Tailwind CSS
- Datos mock gestionados a través de una capa de servicios

- Git y GitHub
- Vercel (Despliegue)

Arquitectura del sistema

El proyecto fue desarrollado bajo una arquitectura modular basada en componentes, siguiendo buenas prácticas de desarrollo frontend moderno.
Se separaron responsabilidades en diferentes capas:

- Interfaz de usuario (UI)
- Lógica de negocio
- Gestión de estado
- Servicios (API)
- Autenticación

Esta organización permite escalar el proyecto, facilitar el mantenimiento y mejorar la reutilización de código.

Gestión de autenticación y roles

El sistema implementa un mecanismo básico de autenticación utilizando almacenamiento en localStorage para mantener la sesión del usuario.

Se manejan dos tipos de roles:

- Gerente: Puede crear, editar y eliminar proyectos, así como asignar tareas a usuarios.
- Usuario: Puede visualizar proyectos asignados y actualizar el estado de sus tareas.

La aplicación utiliza rutas protegidas mediante lógica condicional (isAuthenticated), permitiendo restringir el acceso a ciertas vistas según el estado de sesión.

Simulación de API

El sistema no utiliza una API real ni herramientas como JSON Server. En su lugar, se implementa una capa de servicios que simula el comportamiento de una API REST.
Estas funciones reproducen operaciones como login, registro y gestión de datos, utilizando estructuras en memoria (mocks) y simulando latencia mediante funciones asíncronas. Esto permite mantener una arquitectura similar a una aplicación real sin necesidad de un backend.

Estructura del proyecto

El sistema está organizado siguiendo la estructura típica de un proyecto en Next.js, separando componentes, páginas y servicios.

- pages/: rutas principales de la aplicación.
- components/: componentes reutilizables (UI).
- services/: lógica de consumo de API.

- context/: manejo de autenticación y estado global.
- mocks/: datos simulados.
- styles/: estilos globales.

Esta estructura permite mantener el código limpio, escalable y fácil de mantener.

Flujo del sistema

El sistema sigue un flujo lógico orientado a la gestión de proyectos:

1. El usuario se registra o inicia sesión.
2. El sistema valida las credenciales.
3. Se almacena la sesión en localStorage.
4. El usuario accede al dashboard.
5. Se visualizan los proyectos disponibles.
6. El gerente puede crear o modificar proyectos y tareas.
7. El usuario puede actualizar el estado de sus tareas.
8. Los cambios se reflejan en la interfaz en tiempo real.
9. Los datos se gestionan mediante una capa de servicios que simula el comportamiento de una API REST.

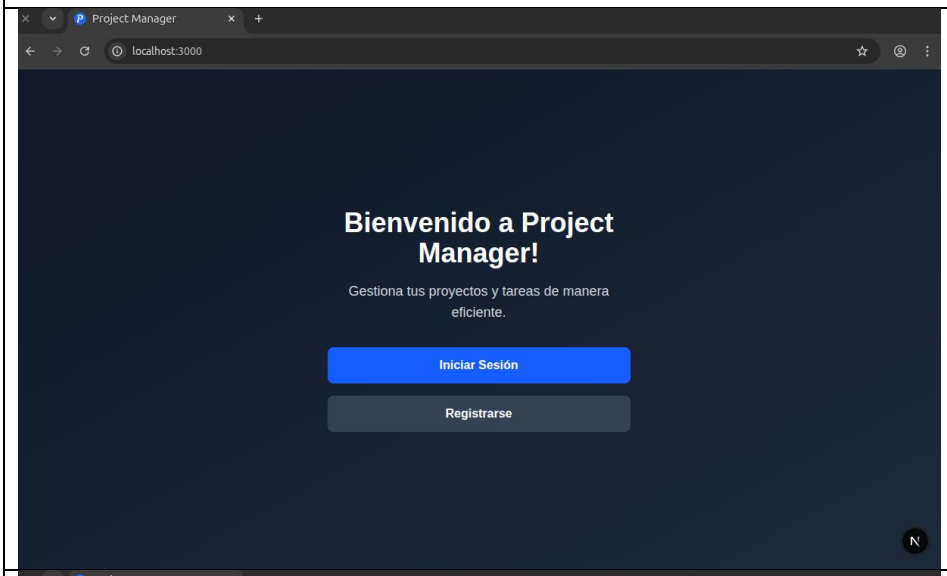
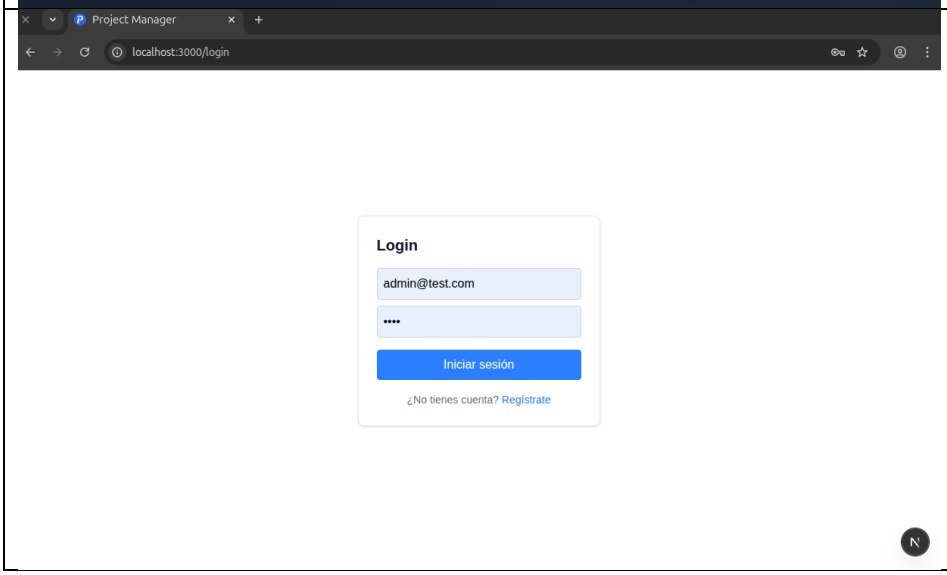
Funcionalidades implementadas

- Autenticación de usuarios.
- Manejo de roles (gerente y usuario).
- Protección de rutas.
- CRUD de proyectos.
- CRUD de tareas.
- Asignación de tareas a usuarios.
- Visualización de progreso.
- Dashboard interactivo.
- Simulación de consumo de API REST mediante servicios.
- Diseño responsivo.
- Uso de control de versiones con Git.

Capturas del sistema

A continuación, se presentan capturas de pantalla de las principales funcionalidades de la aplicación, con el objetivo de evidenciar el funcionamiento del sistema y la interfaz de usuario desarrollada.

Las imágenes incluyen vistas del proceso de autenticación, el dashboard principal, la gestión de proyectos y la administración de tareas, mostrando el comportamiento del sistema según el rol del usuario.

CAPTURA	DESCRIPCION
	Figura 1: Pantalla de inicio de aplicación.
	Figura 2: Pantalla de inicio de sesión

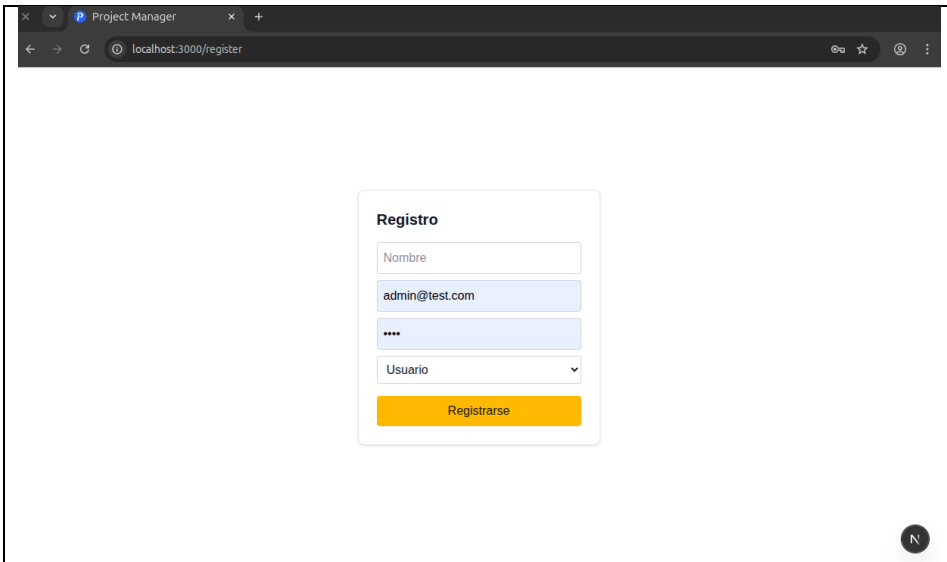


Figura 3: Pantalla de registro para nuevo usuario.

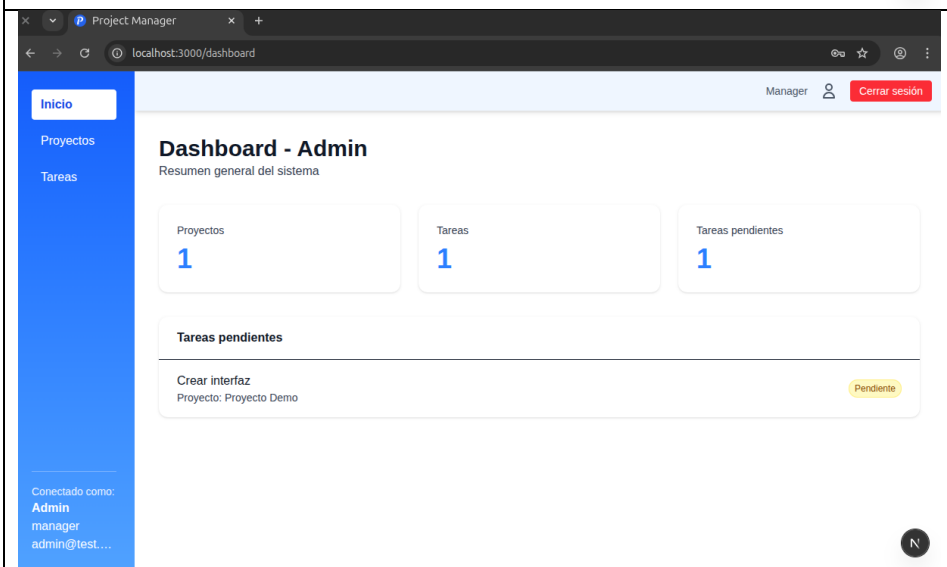


Figura 4: Dashboard principal

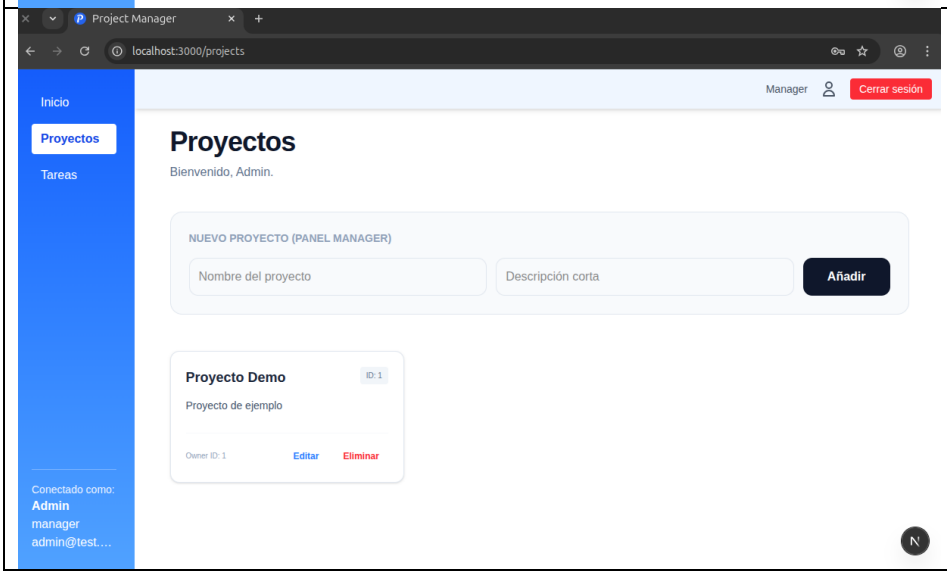


Figura 5: Gestión de proyectos

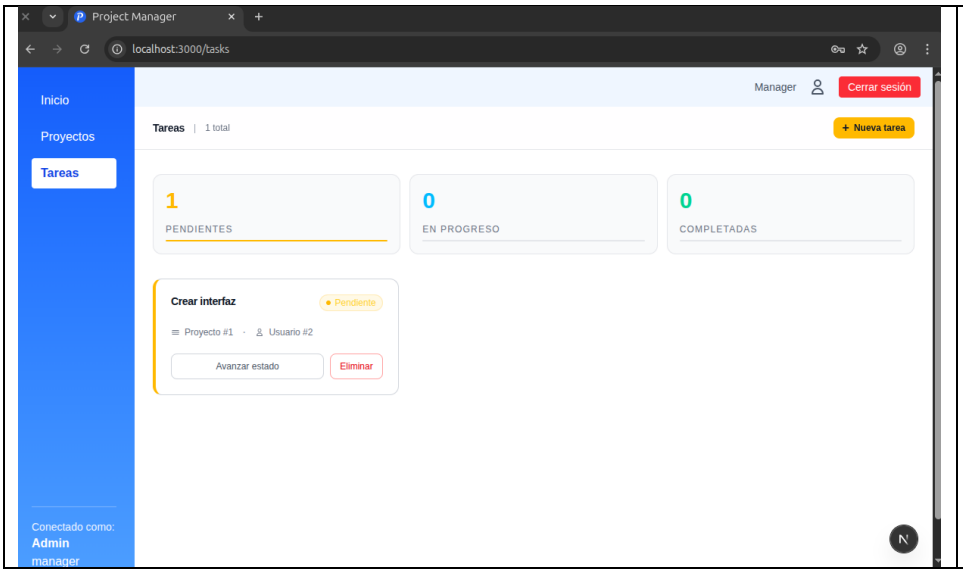


Figura 6: Gestión de tareas

Despliegue y código

Enlace del sistema:
pendiente

Repositorio:
https://github.com/VMz69/project-manager

Organización del equipo

El desarrollo del proyecto se realizó utilizando control de versiones con Git y GitHub, permitiendo un trabajo colaborativo eficiente.
Cada integrante trabajó en una rama independiente, facilitando la integración de cambios y evitando conflictos.
Se aplicaron buenas prácticas de versionamiento y organización del código para mantener un flujo de trabajo ordenado.

Conclusiones

El desarrollo de este proyecto permitió aplicar conocimientos fundamentales del desarrollo web moderno, como el uso de React y Next.js, manejo de estado, consumo de APIs y autenticación básica.
Se logró implementar un sistema funcional de gestión de proyectos con roles diferenciados, lo que permitió simular un entorno real de trabajo colaborativo. Además, se fortalecieron habilidades en la organización del código, uso de herramientas de control de versiones y despliegue de aplicaciones.
Durante el desarrollo se presentaron desafíos relacionados con la gestión del estado, la implementación de autenticación y la integración con la API simulada, los cuales fueron resueltos mediante una adecuada estructuración del proyecto.
Como mejora futura, se recomienda integrar una base de datos real, implementar autenticación segura con backend (JWT), y agregar notificaciones o seguimiento en tiempo real. Esto permitiría escalar la aplicación a un entorno más robusto y profesional.